

Strategic Compression and Edge Deployment of Vision Transformer and CNN Forensic Models on ARM Architectures

1. Introduction and Architectural Overview

The deployment of deep learning-based image forensics models on resource-constrained edge devices represents a profound engineering challenge at the intersection of algorithmic efficiency, precision preservation, and hardware optimization. The overarching objective of this analysis is to define a deployment pathway for a suite of four highly specialized multimedia forensics models—FatFormer, CAT-Net v2, MVSS-Net, and Mesorch—onto a Raspberry Pi 5 platform. The target hardware features an ARM Cortex-A76 processor, 8GB of LPDDR4X RAM, and is augmented with a Hailo-8L Neural Processing Unit (NPU) operating via a PCIe Gen 3 interface, capable of delivering 13 Tera-Operations Per Second (TOPS) of INT8 compute.¹

The primary constraint of this deployment is a strict budget of less than 500MB total storage footprint for all four models combined, paired with a requirement for a cumulative inference pipeline latency of under one second. The uncompressed, native versions of these models collectively exceed 1.26 gigabytes. FatFormer alone, utilizing a Contrastive Language-Image Pre-training (CLIP) ViT-L/14 backbone, occupies approximately 890MB.⁴ CAT-Net v2 requires approximately 150MB for its dual-branch architecture⁶, MVSS-Net demands roughly 120MB for its multi-scale noise analysis⁸, and Mesorch consumes around 100MB for its parallel mesoscopic spatial detection network.¹⁰

Standard computer vision models designed for semantic classification or object detection (such as ResNet or YOLO) rely on macroscopic, high-amplitude visual features. Consequently, they are highly amenable to aggressive pruning and quantization. In stark contrast, digital image forensics models are fundamentally designed to detect microscopic and mesoscopic anomalies. These models analyze Photo Response Non-Uniformity (PRNU) sensor noise, Discrete Cosine Transform (DCT) quantization grid artifacts, subtle blending boundaries, and generative diffusion residuals.⁶ These forensic signals operate at extremely low amplitudes, often residing in the lowest bit-planes of the image tensor.

When traditional, greedy model compression techniques are applied to forensic architectures, the resulting algorithmic noise—such as the quantization noise floor or the loss of high-frequency filters via magnitude pruning—can entirely eclipse the subtle tampering signals, leading to catastrophic degradation in detection accuracy.¹² Therefore, compressing these four

specific models for the Hailo-8L NPU necessitates a highly nuanced, forensics-aware approach to Vision Transformer (ViT) reduction, knowledge distillation, low-bit quantization, and structured multi-stream pruning. This report exhaustively details the methodologies, empirical outcomes, and hardware-specific configurations required to achieve the <500MB and <1s inference parameters without sacrificing forensic integrity.

2. Vision Transformer Compression for Edge Forensics

Vision Transformers have recently revolutionized image forensics by capturing long-range global dependencies and semantic-forgery misalignments that traditional Convolutional Neural Networks (CNNs) often overlook. However, the $\mathcal{O}(N^2 \cdot d)$ computational complexity of the ViT self-attention mechanism—where N represents the number of image patches (tokens) and d represents the embedding dimension—creates prohibitive memory and latency bottlenecks on edge NPUs.¹³ For the FatFormer model, which leverages a massive CLIP ViT-L/14 backbone integrated with a Discrete Wavelet Transform (DWT) Forgery-Aware Adapter, edge deployment is impossible without profound structural intervention.⁴

2.1 ViT Pruning Methodologies

Pruning strategies for Vision Transformers generally fall into three categories: token pruning, head pruning, and layer (depth) pruning. Each interacts differently with the extraction of forensic features.

Token pruning operates on the premise that not all image patches contribute equally to the final prediction. In standard image classification, background tokens are dynamically identified and discarded in early transformer blocks, concentrating computation solely on the salient objects. However, token pruning is highly detrimental to forensic applications. Synthetically generated images, particularly those produced by diffusion models or localized inpainting, often exhibit manipulation artifacts in non-salient background regions, such as inconsistent background blurring, mismatched shadows, or localized upsampling checkerboard patterns.¹⁴ Aggressive token pruning risks permanently discarding the specific patches containing the evidentiary tampering signals. Frameworks like IPCV (Information-Preserving Compression for Vision encoders) attempt to mitigate this by temporarily reconstructing pruned tokens via Neighbor-Guided Reconstruction (NGR) before passing them to the final layers, but this introduces latency overheads counterproductive to the <1s budget.¹⁴

Head pruning and layer pruning offer more predictable acceleration for edge deployment. Head pruning removes redundant multi-head self-attention (MHSA) components. Algorithms such as S2ViTE target sub-modules by evaluating the importance of weights, activations, and gradients, achieving approximately 30% to 34% parameter reduction with a minimal top-1 accuracy drop (less than 1%) on standard benchmarks.¹⁵ Layer pruning reduces the overall

depth of the transformer. While depth reduction is highly hardware-friendly—as it directly reduces the sequential memory bandwidth required during execution—it alters the receptive field of the model. For FatFormer, which relies on the deep semantic alignment of the CLIP model to guide its Forgery-Aware Adapter⁴, layer pruning disrupts the pre-trained language-vision embedding space. Consequently, structured head pruning is preferred over token or layer pruning, provided the importance criterion protects the high-frequency attention heads.

2.2 ViT Quantization: PTQ versus QAT

Quantization maps the 32-bit floating-point (FP32) weights and activations of the ViT into lower-precision formats. Post-Training Quantization (PTQ) applies this mapping after training using a small calibration dataset to determine the optimal scaling factors and zero-points. While PTQ is fast and requires no retraining, it frequently fails when applied to Vision Transformers. ViTs exhibit extreme variations in activation ranges, particularly within the LayerNorm operations and the Softmax outputs of the attention blocks. Uniform PTQ algorithms struggle to capture these long-tailed distributions, leading to severe quantization errors that degrade the model's ability to represent the subtle, localized gradients required for forgery detection.¹²

Quantization-Aware Training (QAT) resolves this by incorporating simulated quantization operations into the forward pass during model fine-tuning. The gradients are approximated through the non-differentiable quantization steps using a Straight-Through Estimator (STE). For ViTs in forensic applications, QAT is mandatory. Research into ViT quantization demonstrates that QAT can recover the accuracy lost to quantization noise, allowing INT8 models to perform within 1% to 2% of their FP32 counterparts.¹⁷ To deploy the FatFormer ViT backbone on the Hailo-8L NPU—which strictly requires 8-bit integer precision for peak performance—an INT8 QAT pipeline must be executed prior to compilation.

2.3 Vision Transformer Distillation

Knowledge Distillation (KD) provides a pathway to train compact ViT architectures by mimicking the behavior of a heavy teacher model. DeiT-style (Data-efficient Image Transformers) distillation introduced a specific distillation token that allows the student transformer to learn from the hard labels of a CNN teacher, transferring the inductive biases of convolutions into the transformer architecture.¹⁹

For extreme edge deployment, architectures such as TinyViT, EfficientViT, and MobileViT (v1/v2/v3) have been designed specifically to replace standard ViT backbones. MobileViT, for instance, integrates MobileNet-style inverted residual blocks with lightweight transformer

blocks, replacing the dense $\mathcal{O}(N^2)$ global attention with localized, efficient attention mechanisms. This dramatically lowers the parameter count to under 20MB while retaining a global receptive field. Distilling a heavy forensic teacher into an EfficientViT or MobileViT student allows the model to process high-resolution images—a critical requirement for

analyzing high-frequency DCT grids—without exceeding the memory or compute constraints of the Raspberry Pi 5.²⁰

2.4 CLIP-Specific Compression and Downstream Accuracy Drops

Because FatFormer utilizes a CLIP ViT-L/14 backbone to establish a generalized semantic space for forgery detection, replacing this backbone requires evaluating lightweight CLIP variants.

Apple's MobileCLIP (2024) utilizes a multi-modal reinforced training approach, transferring knowledge from strong CLIP ensembles and image captioning models into a highly efficient architecture.²² The MobileCLIP-S0 variant achieves zero-shot performance comparable to OpenAI's standard ViT-B/16 while being 4.8× faster and 2.8× smaller, utilizing only 10-15 million parameters for the vision encoder.²³ The upgraded MobileCLIP2-S4 matches the accuracy of massive models like SigLIP-SO400M/14 on ImageNet-1k while maintaining a footprint optimized for mobile latency.²⁴

Similarly, TinyCLIP achieves compression through cross-modal affinity mimicking and weight inheritance, directly distilling both the intermediate representations and the visual-linguistic affinity space of a heavy CLIP teacher into a compact student.²⁵

When utilizing these compressed CLIP models for downstream classification tasks (such as AI-generated image detection), there is an expected accuracy drop compared to the massive ViT-L/14. Empirical evaluations demonstrate that on downstream tasks, MobileCLIP variants generally experience an accuracy drop of 1.5% to 3.0% compared to the 300M+ parameter ViT-L/14, depending on the complexity of the dataset.²⁶ However, this minimal accuracy penalty is offset by a parameter reduction of over 85%, making it an exceptionally favorable trade-off for the <500MB total deployment budget.

Lightweight ViT Alternative	Distillation / Compression Method	Expected Size Reduction vs ViT-L/14	Downstream Accuracy Impact
MobileCLIP-S0	Multi-modal reinforced training KD	~90% reduction (<25MB)	~2.5% to 3.5% drop
MobileCLIP-S2	Multi-modal reinforced training	~82% reduction (~50MB)	~1.0% to 2.0% drop

	KD		
TinyCLIP	Affinity mimicking, weight inheritance	~75% reduction (~75MB)	~1.5% to 2.5% drop
EfficientViT	Linear attention, architectural search	~85% reduction (~45MB)	Highly dependent on fine-tuning

2.5 Critical Analysis: Retraining the FatFormer Forgery-Aware Adapter

FatFormer is designed specifically to detect generalizable synthetic images across unseen GANs and diffusion models. It achieves this not by fine-tuning the entirety of the CLIP ViT-L/14 backbone, but by freezing the backbone and introducing a Forgery-Aware Adapter (FAA) based on the Discrete Wavelet Transform (DWT) alongside a Language-Guided Alignment (LGA) module.⁴ The FAA discerns local forgery traces within both the image and frequency domains, while the LGA module aligns these adapted features with text prompt embeddings to improve generalization.⁴

A critical architectural question is whether the massive ViT-L/14 backbone can be severed, replaced with a MobileCLIP-S2 or OpenCLIP ViT-B/32 backbone, and have *only* the Forgery-Aware Adapter retrained.

The structural design of FatFormer explicitly supports this modularity. The adapter functions by fusing features from intermediate blocks of the vision encoder (e.g., fusing shallow layer features into the adapter via 1×1 convolutions to align dimensions) without interfering with the primary task-agnostic semantic space of the pre-trained CLIP model.²⁹ Ablation studies conducted on the FatFormer architecture have proven that the forgery adaptation mechanism is backbone-agnostic; applying the FAA to alternative backbones such as ViT-B/16, Swin-B, and Swin-L consistently boosts synthetic detection performance across diverse generative models.⁵

Therefore, substituting ViT-L/14 with MobileCLIP-S2 is highly viable. The deployment strategy dictates freezing the MobileCLIP-S2 vision encoder, re-initializing the DWT-based Forgery-Aware Adapter, and retraining the adapter on the target ProGAN or mixed-generator dataset. This approach preserves the robust language-vision alignment of the pre-trained model while reducing the FatFormer memory footprint from ~890MB to approximately

45MB-55MB, easily fitting within the Raspberry Pi 5 budget.

3. Knowledge Distillation for Forensics Models

Knowledge Distillation (KD) provides a mathematically rigorous framework for compressing heavy forensics models into lightweight student architectures. For edge deployment, KD ensures that the lightweight drop-in replacements maintain the specific inductive biases required to detect image manipulation.

3.1 Teacher-Student Distillation Paradigms

In standard computer vision, a teacher model transfers class probabilities to a student model. In the forensic domain, the teacher is a highly parameterized, forensics-specific architecture trained to identify tampering artifacts. Frameworks such as KLMN (Knowledge Distillation based Lightweight Multi-clue Network) demonstrate the efficacy of this approach. KLMN distills multi-clue image forgery detection capabilities from a heavy teacher into a highly efficient student model, achieving superior detection and localization performance while meeting strict requirements for reduced parameters and memory usage.³¹

Similarly, the Reinforced Multi-teacher Knowledge Distillation (Re-MTKD) framework addresses the complexity of modern forgeries by employing multiple teacher models—each specifically trained on distinct manipulation types such as copy-move, splicing, and inpainting.³³ A dynamic teacher selection strategy assigns weights to these teachers during the distillation process, allowing a single, lightweight student model (e.g., a compact ConvNeXt or MobileNet variant) to effectively learn the commonalities and specific traces of diverse tampering operations.³³

3.2 Cross-Architecture Distillation

Deploying to edge NPUs like the Hailo-8L often favors convolutional operations, which are highly optimized in hardware accelerators compared to the dynamic memory access patterns of transformer attention maps. Cross-architecture distillation—such as distilling a ViT teacher into a CNN student—bridges this gap.

By utilizing a ViT teacher (which excels at capturing global semantic inconsistencies, such as a mismatched reflection or lighting error across an image) to guide a CNN student (which excels at local, high-frequency artifact extraction), the CNN student inherits a pseudo-global receptive field.³⁵ This allows a model like MobileNetV3-Small to output localization masks that account for both local pixel anomalies and global context, maximizing the inference speed on the Raspberry Pi 5 without the overhead of computing MHSA matrices.

3.3 Feature, Logit, and Attention-Level Distillation

The specific mechanism of distillation dictates how well forensic signals are transferred.

- **Logit-Level Distillation:** Response-based KD forces the student to match the final

classification logits (the soft targets) of the teacher.²⁰ While effective for binary authentication (real vs. fake), logit distillation fails to provide the spatial gradients necessary for pixel-level forgery localization. It does not teach the student *how* the teacher found the forgery, only *what* the final decision was.

- **Feature-Level Distillation:** For forensics, feature-level distillation is paramount. This technique minimizes the distance (e.g., Mean Squared Error or Cosine Similarity) between the intermediate activation maps of the teacher and the student.³¹ For models like MVSS-Net and CAT-Net, forcing the student to replicate the intermediate feature maps ensures that the student network learns to extract the high-frequency components that contain sensor pattern noise and block artifacts.³⁷
- **Attention-Level Distillation:** When compressing hybrid models like Mesorch, transferring the attention maps from the teacher to the student ensures that the student network learns exactly *where* to focus its receptive field to find mesoscopic boundary inconsistencies, preserving the model's ability to orchestrate multi-scale information.¹⁰

3.4 Preservation of PRNU and DCT Signals

A pivotal concern regarding KD in forensics is whether the distillation process can preserve ultra-subtle forensic signals such as Photo Response Non-Uniformity (PRNU) noise and Discrete Cosine Transform (DCT) artifacts. PRNU acts as a microscopic camera fingerprint extracted via denoising filters, while DCT inconsistencies indicate double-compression ghosts.¹¹

Research confirms that KD can preserve these signals, provided the distillation loss function is properly configured. If KD relies solely on semantic logit matching, the student network will prioritize macroscopic textures and ignore the sub-pixel noise, effectively discarding the PRNU and DCT signals. However, by explicitly implementing feature-level distillation on the layers immediately following the noise-extraction blocks (e.g., the SRM filter output in MVSS-Net or the binary volume output in CAT-Net), the student is mathematically forced to reconstruct the frequency-domain analysis of the teacher.³¹ Implementations like KLMN successfully prove that a compact student can maintain sensitivity to DCT artifacts and noise residue patterns if the intermediate frequency representations are strictly aligned during the KD phase.³¹

4. Quantization Constraints in Forensic Models

To utilize the Hailo-8L NPU and achieve the <1s latency budget on the Raspberry Pi 5, models must be compiled into the Hailo Executable Format (HEF). This compilation process inherently relies on 8-bit integer (INT8) quantization to maximize the efficiency of the NPU's MAC (Multiply-Accumulate) arrays.¹

4.1 INT8 and INT4 Quantization Impact

Linear quantization maps a continuous floating-point range to a discrete integer space. This mathematical transformation introduces a quantization error, or noise floor, determined by the

quantization step size. In standard object detection or semantic segmentation, tensors possess large magnitude variations corresponding to visual edges and colors; thus, the absolute quantization noise is small relative to the signal, and accuracy retention is high.

In digital image forensics, the signal-to-noise ratio is inverted. The core forensic signals—such as the subtle pixel value variations of PRNU or the statistical anomalies of a DCT coefficient histogram—are inherently low-magnitude, high-frequency noise patterns.¹² When these signals are quantized to INT8, the quantization step size can become larger than the amplitude of the forensic signal itself. The algorithmic noise introduced by the INT8 conversion effectively buries the physical noise artifacts the model is attempting to detect, transforming the subtle forensic clues into uniform blocks of zero-values or clipping them entirely.¹²

The industry push toward 4-bit (W4A4) quantization, driven largely by the need to compress Large Language Models (LLMs), is categorically unsuited for image forensics.⁴⁰ While INT4 doubles memory efficiency, it reduces the representational space to just 16 discrete values. Research evaluating INT4 quantization on highly sensitive vision tasks reveals severe degradation in spatial gradients and localized feature extraction, resulting in massive accuracy drops for pixel-level anomaly localization.⁴⁰ Consequently, INT4 quantization must be strictly avoided for the CAT-Net, MVSS-Net, and Mesorch backends.

4.2 Sensitivity of Forensic Models versus Standard Classifiers

Forensic models are orders of magnitude more sensitive to quantization than standard image classifiers. For example, CAT-Net explicitly analyzes the distribution of Y-channel DCT coefficients to identify double-compression anomalies.⁶ If the feature maps representing these statistical distributions are clamped into an INT8 space, the minute statistical outliers that indicate a spliced region are flattened into the surrounding bins. Similarly, MVSS-Net utilizes a noise-sensitive branch initialized with BayarConv or Sobel filters to extract prediction residuals.⁸ The gradients required to maintain these constrained convolutional filters are highly sensitive to low-precision formats; quantization disrupts the strict mathematical constraints of the filters, degrading the network's specificity against authentic images.⁸

4.3 PTQ vs. QAT for Forensic Detection

Given this extreme sensitivity, Post-Training Quantization (PTQ) is generally insufficient for forensic architectures. PTQ calibration relies on statistical observations (e.g., Min-Max or KL-Divergence) of activation tensors to set scaling factors.⁴³ Because forensic anomalies are rare, localized, and low-magnitude, PTQ calibration often treats them as statistical outliers and truncates them.

Quantization-Aware Training (QAT) is critically required. During QAT, the network's weights are fine-tuned while simulating the INT8 precision limits in the forward pass.¹⁸ This allows the training optimizer to adjust the weights dynamically, actively learning to extract the PRNU and

DCT signals within the constraints of the discretized integer space. Studies demonstrate that QAT consistently outperforms PTQ in preserving performance under low-precision constraints for sensitive architectures, restoring the accuracy gap to within acceptable margins.¹⁸

4.4 Toolchains and Mixed-Precision Strategies

The optimization pipeline on the Raspberry Pi 5 relies on the Hailo Dataflow Compiler (DFC), which ingests ONNX or TFLite models and outputs the optimized HEF binary.⁴⁶ Unlike TensorRT, which allows dynamic precision swapping at runtime based on layer profiling, the Hailo DFC requires explicit quantization profiles defined during the compilation stage.⁴⁷

To mitigate quantization destruction in forensics models, a mixed-precision strategy must be defined within the Hailo DFC environment.

1. **Preserve Input Filters in FP16/BF16:** The initial layers responsible for noise extraction—such as the SRM filter initialization in MVSS-Net, the DWT module in the FatFormer adapter, and the first convolutional layers processing the DCT quantization tables in CAT-Net—must be strictly maintained in FP16 precision.⁸
2. **INT8 for Semantic Aggregation:** The deeper layers of these networks, which aggregate the extracted high-frequency features into mesoscopic or macroscopic spatial representations (e.g., the deep fusion stages of CAT-Net or the Transformer blocks of Mesorch), exhibit higher magnitudes and are significantly more robust to quantization.¹² These layers can be safely quantized to INT8 to leverage the maximum TOPS throughput of the Hailo-8L.

5. Structured Pruning for Dual-Branch Architectures

Forensic models frequently rely on multi-stream architectures to analyze orthogonal data domains concurrently. CAT-Net v2, representing a ~150MB footprint, operates via a dual-branch structure: an RGB stream for visual artifacts and a DCT stream for JPEG compression artifacts.⁶

5.1 Pruning the DCT or RGB Streams in CAT-Net

To meet the <500MB budget, a naive approach might suggest entirely pruning one of CAT-Net's streams, theoretically reducing the parameter count by up to 50%. However, this is fundamentally flawed in the context of generalized image forensics.

The dual streams are not redundant; they are complementary.

- **The RGB Stream:** Analyzes raw pixel data to identify visual inconsistencies, such as unnatural blending boundaries, contrast errors, and spatial splicing edges.
- **The DCT Stream:** Specifically engineered to learn compression artifacts by transforming Y-channel DCT coefficients into a binary volume representation alongside the Y-channel quantization table.⁶ This allows the network to track double-compressed clues,

differentiating between a spliced region (which often behaves as singly compressed) and an authentic background (which exhibits double compression ghosts).⁶

Ablation studies on dual-stream forensic networks demonstrate that dropping the DCT stream causes the model to fail entirely on highly compressed JPEG forgeries, as the visual edges in the RGB domain are smoothed out and obscured by the compression process.⁴⁸ Conversely, dropping the RGB stream renders the model completely blind to forgeries in uncompressed imagery (e.g., PNG or RAW formats), as the DCT stream relies entirely on the presence of block-artifact discrepancies.⁴⁸ Consequently, neither stream can be ablated without severely crippling the model's robustness and cross-domain generalization. Both streams must be retained.

5.2 Channel Pruning and Sensitivity Analysis

Instead of macro-architectural ablation, structured channel pruning must be applied uniformly across both the RGB and DCT streams. Structured pruning removes entire convolutional filters or channels, physically reducing the size of the weight tensors and seamlessly accelerating inference on the Hailo-8L NPU without requiring specialized sparse-matrix hardware support.⁴⁹

The sensitivity of forensic CNNs to channel pruning varies by depth. The early layers of the DCT stream, which utilize 8-dilated convolutions to extract frequency patterns from the binary volume representation⁶, are highly sensitive to pruning. Removing filters here destroys the fundamental feature extractors. Conversely, the deeper layers and the final fusion stage (where the multi-resolution feature maps are bilinearly upsampled and concatenated) contain high levels of parameter redundancy and can be pruned aggressively.⁶

5.3 Filter Importance Criteria

The selection of the filter importance criterion is critical.

- **L1-Norm (Magnitude) Pruning:** This standard technique assumes that filters with small absolute weight values are unimportant and can be discarded. For forensics, this is highly destructive. Low-magnitude weights often encode the subtle, high-pass filters necessary for noise extraction; dropping them removes the model's sensitivity to PRNU and edge anomalies.⁵⁰
- **Taylor Expansion Pruning:** This criterion evaluates the ultimate impact of a filter on the final loss function by utilizing the first-order Taylor expansion (the gradient multiplied by the weight). This ensures that filters crucial to minimizing the forensic classification loss are preserved, regardless of their absolute magnitude.
- **Geometric Median Pruning:** This method identifies and removes filters that are highly similar to other filters within the same layer, preserving geometrically unique filters. For the DCT stream, which must detect a wide variety of quantization table discrepancies, Geometric Median pruning successfully preserves the diverse filter bank required to detect rare compression artifacts while eliminating structural redundancy.⁵⁰ Taylor

Expansion and Geometric Median are the strongly recommended criteria for compressing CAT-Net v2.

6. ARM and Raspberry Pi 5 Specific Optimizations

Deploying the compressed forensic pipeline onto the Raspberry Pi 5 requires strict adherence to system-level constraints and hardware-accelerator compilers specific to the ARM Cortex-A76 and Hailo-8L ecosystem.

6.1 Inference Frameworks: PyTorch Mobile, ONNX, and TFLite

The Raspberry Pi 5 processor (Cortex-A76) supports NEON SIMD (Single Instruction, Multiple Data) instructions, which optimize vectorized matrix math for deep learning.²

- **PyTorch Mobile:** While natively compatible with research codebases, PyTorch Mobile introduces significant runtime overhead and memory bloat on ARM devices, making it suboptimal for a rapid sequential pipeline.
- **TensorFlow Lite (TFLite):** TFLite is highly optimized for ARM NEON SIMD and offers excellent XNNPACK delegates for CPU acceleration. However, mapping complex PyTorch forensic models (like Mesorch) through the TFLite converter often results in unsupported operation errors.
- **ONNX Runtime:** ONNX Runtime provides the most robust execution graph for the Raspberry Pi 5. It natively supports the complex multi-stream graphs of CAT-Net and the hybrid CNN-Transformer architectures of Mesorch. Crucially, the Hailo Dataflow Compiler fundamentally relies on ONNX as its primary ingestion format.⁴⁶ Therefore, ONNX Runtime is the required framework for the host CPU, seamlessly handing off compiled sub-graphs to the Hailo NPU.

6.2 Hailo-8L AI HAT Pipeline and Benchmarks

The Hailo-8L AI HAT+ interfaces with the Pi 5 via PCIe Gen 3, a mandatory configuration to prevent bus bottlenecking (Gen 3 allows ~76 FPS for standard streams compared to ~39 FPS on Gen 2).² The NPU provides 13 TOPS of INT8 performance.³

The compilation pipeline requires parsing the ONNX model through the Hailo Dataflow Compiler (DFC) to generate a Hailo Executable Format (HEF) file.⁴⁶ The DFC supports a broad range of operations, including standard convolutions, Mish, Hard-Swish, SiLU, and basic attention mechanisms.⁵³ However, highly specific forensic operations—such as the Discrete Wavelet Transform (DWT) utilized in the FatFormer adapter or custom Spatial Rich Model (SRM) filters—may not map natively to the NPU's ALU array. The Hailo compiler handles this by partitioning the graph: unsupported high-frequency extractions are executed as fallback operations on the Pi 5's Cortex-A76 CPU via ONNX Runtime, while the heavy semantic processing is offloaded to the Hailo-8L.

Published benchmarks for the Raspberry Pi 5 + Hailo-8L validate its capability to meet the <1s latency budget. Standard CNNs like YOLOv8s execute at ~127 FPS, while SSD-MobileNetV2 processes at ~145 FPS.⁵⁴ While ViT architectures are slower, lightweight transformers (e.g., MobileCLIP variants) optimized through the Hailo DFC comfortably operate at >30 FPS, ensuring the entire 4-model cascade executes well within the 1000ms threshold.

6.3 Memory Management and Model Loading Strategies

The target budget requires executing four distinct models sequentially:

1. FatFormer (AI generation check)
2. CAT-Net v2 (JPEG compression check)
3. MVSS-Net (Noise manipulation check)
4. Mesorch (Spatial localization).

The Raspberry Pi 5 is equipped with 8GB of RAM, which is vastly superior to the <500MB total size of the compressed models. Memory *capacity* is not the bottleneck; memory *bandwidth* and context switching are.¹

The Hailo-8L executes multiple HEF contexts sequentially. Loading a HEF file from the Raspberry Pi's microSD card to the NPU on-demand incurs massive I/O latency, effectively destroying the <1s inference budget.⁵⁶

Loading Strategy: All four compiled HEF models must be loaded entirely into the Raspberry Pi's RAM at system startup. During runtime, the HailoRT middleware keeps the NPU initialized. To mitigate the PCIe context-switching overhead of swapping between the four models on the NPU, inference requests should be batched (e.g., batch size 4 or 8) across the forensic pipeline, passing multiple images through FatFormer before switching the NPU context to CAT-Net.⁵⁶

7. Lightweight Alternatives and Drop-in Replacements

To aggressively meet the <500MB budget, the heavy backbones of MVSS-Net and Mesorch must be swapped for natively efficient architectures prior to knowledge distillation.

7.1 CNN Backbone Replacements (MVSS-Net)

MVSS-Net originally utilizes two heavy ResNet-50 backbones for its dual branches, resulting in a ~120MB footprint.⁸

- **MobileNetV3-Small:** By utilizing depthwise separable convolutions, hard-swish activations, and squeeze-and-excitation blocks, MobileNetV3-Small provides an ideal drop-in replacement.⁵⁷ It reduces the parameter count from ~25M to under 3M per branch, shrinking MVSS-Net to approximately 25MB.
- **EfficientNet-Lite:** Offers slightly higher accuracy than MobileNetV3 at a marginal size increase, removing the complex squeeze-and-excitation blocks to maximize hardware

compatibility on edge devices. Distilling the original MVSS-Net into a MobileNetV3-Small student preserves the multi-scale edge and noise perception while guaranteeing high FPS execution on the Hailo-8L.³¹

7.2 Spatial Localization Replacements (Mesorch)

Mesorch (~100MB) operates at the mesoscopic level, fusing CNNs (for micro details) and Transformers (for macro information) across parallel scales.¹⁰

- **Mesorch-P (Pruned):** The original researchers proposed Mesorch-P, which dynamically drops low-significance scales from the multi-scale hierarchy. This scale pruning reduces the parameters from 85.7M to 62.2M, decreasing FLOPs dramatically while maintaining state-of-the-art F1 scores and robustness to perturbations.¹⁰
- **Lightweight Segmentation Backends:** To compress further, the CNN feature extractor within Mesorch can be replaced with Fast-SCNN or BiSeNet-v2, which are designed specifically for high-resolution, real-time spatial segmentation at the edge. Integrating these backends can compress the model to ~50MB.

7.3 Size-Accuracy Trade-off Summary for Drop-in Replacements

Original Architecture	Lightweight Replacement	Target Task	Expected Size	Expected Accuracy Trade-off
CLIP ViT-L/14	MobileCLIP-S2	AI Generation Det.	~50MB	-1.5% to -2.0% zero-shot accuracy ²⁶
ResNet-50 (Dual)	MobileNetV3-Small	Noise Analysis	~25MB	Slight localization boundary softening ⁵⁹
Mesorch Standard	Mesorch-P + Fast-SCNN	Spatial Localization	~50MB	Negligible F1 impact due to adaptive scale weighting ¹⁰

8. Final Compression Outcomes and Deployment Matrix

Applying the synthesized methodologies—backbone replacement, Knowledge Distillation (KD),

Quantization-Aware Training (QAT), and structured Taylor pruning—yields a comprehensive deployment blueprint. This blueprint successfully reduces the 1.26GB baseline to an estimated 180MB, definitively satisfying the <500MB storage constraint and enabling <1s sequential inference on the Raspberry Pi 5 and Hailo-8L.

Model	Original Size	Optimal Compression Method & Architecture Adjustments	Expected Target Size	Expected Accuracy Impact
FatFormer	~890MB	Backbone Swap & Retrain: Replace the massive ViT-L/14 with MobileCLIP-S2. Freeze the backbone and retrain <i>only</i> the DWT Forgery-Aware Adapter on the specific dataset. Apply INT8 QAT. ⁴	~50MB	Minimal (~1-2.5% drop). The retraining of the adapter recovers the forensic sensitivity lost by shrinking the global semantic backbone. ⁵
CAT-Net v2	~150MB	Structured Pruning: Apply Taylor-expansion or Geometric Median channel pruning equally to both RGB and DCT streams. Do <i>not</i> ablate either stream	~55MB	~2-3% drop in F1 score. Explicitly preserves the model's ability to detect double-compression ghosts. ⁴⁸

		entirely. Apply INT8 QAT, retaining FP16 for initial quantization table parsing. ⁶		
MVSS-Net	~120MB	KD & Architecture Swap: Replace the dual ResNet-50 branches with MobileNetV3-Small. Use feature-level Knowledge Distillation from the original network to preserve noise/Sobel extraction. Compile with INT8 QAT. ⁸	~25MB	Negligible image-level detection drop; minor boundary softening on pixel-level masks offset by feature KD. ³¹
Mesorch	~100MB	Scale Pruning: Implement the Mesorch-P variant, which adaptively drops low-significance multi-scale layers. Apply INT8 QAT for the CNN components. ¹⁰	~50MB	Extremely low impact. Mesorch-P explicitly maintains state-of-the-art robustness and F1 metrics against standard baselines. ¹⁰
TOTAL	~1.26 GB		~180MB	Constraint Satisfied.

Through rigorous adherence to forensic-specific quantization sensitivities, cross-architecture knowledge distillation, and the hardware specificities of the Hailo Dataflow Compiler, the integration of these four sophisticated models into a unified, high-speed edge pipeline is highly viable.

참고 자료

1. Introducing the Raspberry Pi AI HAT+ 2: Generative AI on Raspberry Pi 5, 3월 19, 2026에 액세스, <https://www.raspberrypi.com/news/introducing-the-raspberry-pi-ai-hat-plus-2-generative-ai-on-raspberry-pi-5/>
2. Part 1 - Raspberry Pi From Zero to Hero Course - Hackster.io, 3월 19, 2026에 액세스, <https://www.hackster.io/skruglewicz/part-1-raspberry-pi-from-zero-to-hero-course-7a855d>
3. Hailo-8L Entry-Level AI Accelerator Solutions, 3월 19, 2026에 액세스, <https://hailo.ai/products/ai-accelerators/hailo-8l-ai-accelerator-for-ai-light-applications/>
4. Forgery-aware Adaptive Transformer for Generalizable Synthetic Image Detection - CVPR, 3월 19, 2026에 액세스, <https://cvpr.thecvf.com/virtual/2024/poster/31740>
5. Forgery-aware Adaptive Transformer for Generalizable Synthetic Image Detection - CVF Open Access, 3월 19, 2026에 액세스, https://openaccess.thecvf.com/content/CVPR2024/papers/Liu_Forgery-aware_Adaptive_Transformer_for_Generalizable_Synthetic_Image_Detection_CVPR_2024_paper.pdf
6. CAT-Net: Compression Artifact Tracing Network for Detection and Localization of Image Splicing - CVF Open Access, 3월 19, 2026에 액세스, https://openaccess.thecvf.com/content/WACV2021/papers/Kwon_CAT-Net_Compression_Artifact_Tracing_Network_for_Detection_and_Localization_of_WACV_2021_paper.pdf
7. Multi-branch network for double JPEG detection and localization - PMC, 3월 19, 2026에 액세스, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12134093/>
8. (PDF) MVSS-Net: Multi-View Multi-Scale Supervised Networks for Image Manipulation Detection - ResearchGate, 3월 19, 2026에 액세스, https://www.researchgate.net/publication/361037074_MVSS-Net_Multi-View_Multi-Scale_Supervised_Networks_for_Image_Manipulation_Detection
9. AMSEANet: An Edge-Guided Adaptive Multi-Scale Network for Image Splicing Detection and Localization - PMC, 3월 19, 2026에 액세스, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12568058/>
10. Mesoscopic Insights: Orchestrating Multi-Scale & Hybrid Architecture for Image Manipulation Localization - AAAI.org, 3월 19, 2026에 액세스, <https://ojs.aaai.org/index.php/AAAI/article/view/33198/35353>
11. ForensicFormer: Hierarchical Multi-Scale Reasoning for Cross-Domain Image Forgery Detection - arXiv, 3월 19, 2026에 액세스,

- <https://arxiv.org/html/2601.08873v1>
12. Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference, 3월 19, 2026에 액세스, https://www.researchgate.net/publication/329740169_Quantization_and_Training_of_Neural_Networks_for_Efficient_Integer-Arithmetic-Only_Inference
 13. CVPR Poster DeepCompress-ViT: Rethinking Model Compression to Enhance Efficiency of Vision Transformers at the Edge, 3월 19, 2026에 액세스, <https://cvpr.thecvf.com/virtual/2025/poster/33707>
 14. SparseViT: Revisiting Activation Sparsity for Efficient High-Resolution Vision Transformer | Request PDF - ResearchGate, 3월 19, 2026에 액세스, https://www.researchgate.net/publication/373309740_SparseViT_Revisiting_Activation_Sparsity_for_Efficient_High-Resolution_Vision_Transformer
 15. Vision Transformers on the Edge: A Comprehensive Survey of Model Compression and Acceleration Strategies - arXiv, 3월 19, 2026에 액세스, <https://arxiv.org/html/2503.02891v1>
 16. Vision Transformers on the Edge: A Comprehensive Survey of Model Compression and Acceleration Strategies - arXiv, 3월 19, 2026에 액세스, <https://arxiv.org/pdf/2503.02891>
 17. PoMQ-ViT: Mixed-Precision Quantization Vision Transformer with Pareto Optimization, 3월 19, 2026에 액세스, <https://www.mdpi.com/2076-3417/15/18/9856>
 18. Systematic Review of Quantization-Optimized Lightweight Transformer Architectures for Real-Time Fruit Ripeness Detection on Edge Devices - MDPI, 3월 19, 2026에 액세스, <https://www.mdpi.com/2073-431X/15/1/69>
 19. ZeBROD: Zero-Retraining Based Recognition and Object Detection Framework - arXiv.org, 3월 19, 2026에 액세스, <https://arxiv.org/pdf/2512.04888>
 20. NeurIPS Poster Taught Well Learned III: Towards Distillation-conditional Backdoor Attack, 3월 19, 2026에 액세스, <https://neurips.cc/virtual/2025/poster/118829>
 21. MobileViCLIP: An Efficient Video-Text Model for Mobile Devices - arXiv.org, 3월 19, 2026에 액세스, <https://arxiv.org/html/2508.07312v1>
 22. MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training, 3월 19, 2026에 액세스, <https://machinelearning.apple.com/research/mobileclip>
 23. Ultralytics implementation of the research paper, "MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training" CVPR 2024 - GitHub, 3월 19, 2026에 액세스, <https://github.com/ultralytics/mobileclip>
 24. MobileCLIP2: Improving Multi-Modal Reinforced Training, 3월 19, 2026에 액세스, <https://machinelearning.apple.com/research/mobileclip2>
 25. TinyCLIP: CLIP Distillation via Affinity Mimicking and Weight Inheritance - CVF Open Access, 3월 19, 2026에 액세스, https://openaccess.thecvf.com/content/ICCV2023/papers/Wu_TinyCLIP_CLIP_Distillation_via_Affinity_Mimicking_and_Weight_Inheritance_ICCV_2023_paper.pdf
 26. MobileCLIP: Fast Image-Text Models through Multi-Modal Reinforced Training - CVF, 3월 19, 2026에 액세스, https://openaccess.thecvf.com/content/CVPR2024/papers/Vasu_MobileCLIP_Fast_Image-Text_Models_through_Multi-Modal_Reinforced_Training_CVPR_2024_paper.pdf

27. MobileCLIP2: Improving Multi-Modal Reinforced Training - arXiv, 3월 19, 2026에 액세스, <https://arxiv.org/html/2508.20691v1>
28. Michel-liu/FatFormer: [CVPR 2024] The official repo for Forgery-aware Adaptive Transformer for Generalizable Synthetic Image Detection - GitHub, 3월 19, 2026에 액세스, <https://github.com/Michel-liu/FatFormer>
29. CVPR Poster Forensics Adapter: Adapting CLIP for Generalizable Face Forgery Detection, 3월 19, 2026에 액세스, <https://cvpr.thecvf.com/virtual/2025/poster/33184>
30. Forensic-MoE: Exploring Comprehensive Synthetic Image Detection Traces with Mixture of Experts, 3월 19, 2026에 액세스, https://openaccess.thecvf.com/content/ICCV2025/supplemental/Fang_Forensic-MoE_Exploring_Comprehensive_ICCV_2025_supplemental.pdf
31. KLMN: Knowledge distillation based lightweight multi-clue image forgery detection and localization - IEEE Xplore, 3월 19, 2026에 액세스, <https://ieeexplore.ieee.org/document/10888764/>
32. Shengyong Chen's research works | Tianjin University of Technology and other places, 3월 19, 2026에 액세스, <https://www.researchgate.net/scientific-contributions/Shengyong-Chen-13436423>
33. Reinforced Multi-teacher Knowledge Distillation for Efficient General Image Forgery Detection and Localization - arXiv.org, 3월 19, 2026에 액세스, <https://arxiv.org/html/2504.05224v1>
34. Reinforced Multi-teacher Knowledge Distillation for Efficient General Image Forgery Detection and Localization, 3월 19, 2026에 액세스, <https://ojs.aaai.org/index.php/AAAI/article/view/32085/34240>
35. Knowledge Distillation in Visual Algorithms: A Survey - 计算机研究与发展, 3월 19, 2026에 액세스, <https://crad.ict.ac.cn/en/article/doi/10.7544/issn1000-1239.202440694>
36. Symmetrical Learning and Transferring: Efficient Knowledge Distillation for Remote Sensing Image Classification - MDPI, 3월 19, 2026에 액세스, <https://www.mdpi.com/2073-8994/17/7/1002>
37. Code-in-the-Loop Forensics: Agentic Tool Use for Image Forgery Detection - arXiv.org, 3월 19, 2026에 액세스, <https://arxiv.org/html/2512.16300v1>
38. Mesoscopic Insights: Orchestrating Multi-Scale & Hybrid Architecture for Image Manipulation Localization | Request PDF - ResearchGate, 3월 19, 2026에 액세스, https://www.researchgate.net/publication/390710849_Mesoscopic_Insights_Orchestrating_Multi-Scale_Hybrid_Architecture_for_Image_Manipulation_Localization
39. Image Representation-Driven Knowledge Distillation for Improved Time-Series Interpretation on Wearable Sensor Data - PMC, 3월 19, 2026에 액세스, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12567629/>
40. A Comprehensive Evaluation on Quantization Techniques for Large Language Models, 3월 19, 2026에 액세스, <https://arxiv.org/html/2507.17417v1>
41. Understanding INT4 Quantization for Language Models: Latency Speedup, Composability, and Failure Cases - arXiv.org, 3월 19, 2026에 액세스, <https://arxiv.org/pdf/2301.12017>

42. Systematic Analysis of Low-Precision Training in Deep Neural Networks: Factors Influencing Matrix Computations - MDPI, 3월 19, 2026에 액세스, <https://www.mdpi.com/2076-3417/14/21/10025>
43. Quantization - Diagnosing layer sensitivity during post training quantization - Embedl Hub, 3월 19, 2026에 액세스, <https://hub.embedl.com/blog/diagnosing-layer-sensitivity>
44. naver-ai/StatAssist-GradBoost: A Study on Optimal INT8 Quantization-aware Training from Scratch - GitHub, 3월 19, 2026에 액세스, <https://github.com/naver-ai/StatAssist-GradBoost>
45. The Impact of 8- and 4-Bit Quantization on the Accuracy and Silicon Area Footprint of Tiny Neural Networks - MDPI, 3월 19, 2026에 액세스, <https://www.mdpi.com/2079-9292/14/1/14>
46. Hailo AI Platform Guide - Hailo Dataflow Compiler - RidgeRun Developer Wiki: Embedded Linux, Multimedia, & Robotics Resources, 3월 19, 2026에 액세스, https://developer.ridgerun.com/wiki/index.php/Hailo/Hailo-8/AI_Software_and_Tools/Hailo_Dataflow_Compiler
47. hailo_model_zoo/docs/GETTING_STARTED.rst at master - GitHub, 3월 19, 2026에 액세스, https://github.com/hailo-ai/hailo_model_zoo/blob/master/docs/GETTING_STARTED.rst
48. arXiv:2311.14218v2 [cs.CV] 31 Mar 2024, 3월 19, 2026에 액세스, <https://arxiv.org/pdf/2311.14218>
49. Adaptive Compression of Large Vision Models for Efficient Image Quality Assessment of AI-Generated Content - IEEE Computer Society, 3월 19, 2026에 액세스, <https://www.computer.org/csdl/proceedings-article/iccvw/2025/898800d140/2e1di2Kmual>
50. Parameter Pruning and Quantization of Image Classification Network - MATLAB & Simulink, 3월 19, 2026에 액세스, <https://www.mathworks.com/help/deeplearning/ug/parameter-pruning-and-quantization-of-image-classification-network.html>
51. Benchmark of Multistream Inference on Raspberrypi 5 with Hailo8 - Seeed Studio, 3월 19, 2026에 액세스, https://wiki.seeedstudio.com/benchmark_of_multistream_inference_on_raspberrypi5_with_hailo8/
52. Hailo Dataflow Compiler User Guide 3.30.0 | PDF - Scribd, 3월 19, 2026에 액세스, <https://www.scribd.com/document/850794798/hailo-dataflow-compiler-v3-30-0-user-guide>
53. Supported operators - General Discussion - Hailo Community, 3월 19, 2026에 액세스, <https://community.hailo.ai/t/supported-operators/5046>
54. Raspberry pi 5 with Hailo-8L Benchmark - General Discussion, 3월 19, 2026에 액세스, <https://community.hailo.ai/t/raspberry-pi-5-with-hailo-8l-benchmark/746>
55. Raspberry Pi AI Kit Setup: Run Local AI Models on Pi 5 - Zbotic, 3월 19, 2026에 액세스, <https://zbotic.in/raspberry-pi-ai-kit-setup-run-local-ai-models-on-pi-5/>
56. Help Needed: Issues with Detection Performance and Multi-Context HEF on Hailo

Device - General Discussion, 3월 19, 2026에 액세스,

<https://community.hailo.ai/t/help-needed-issues-with-detection-performance-and-multi-context-hef-on-hailo-device/6511>

57. Image Classification on Edge Device: MobileNet and Pi 5 - Zbotic, 3월 19, 2026에 액세스,

<https://zbotic.in/image-classification-on-edge-device-mobilenet-and-pi-5/>

58. Mesoscopic Insights: Orchestrating Multi-scale & Hybrid Architecture for Image Manipulation Localization - arXiv.org, 3월 19, 2026에 액세스,

<https://arxiv.org/html/2412.13753v1>

59. Raspberry Pi Object detection with Faster R-CNN - YouTube, 3월 19, 2026에 액세스, <https://www.youtube.com/watch?v=HrcvwPfCUc8>